Django Complete Tutorial Notes (Part 1–8 + Reusable Apps + First Contribution)

Language: English

Level: Beginner to Intermediate

Table of Contents

1. Django Introduction
2. Part 1 — Project, App, Views and URLs
3. Part 2 — Database, Models, ORM and Admin
4. Part 3 — Views and Templates
5. Part 4 — Forms, Voting and Generic Views
6. Part 5 — Automated Testing
7. Part 6 — Static Files
8. Part 7 — Django Admin Customization
9. Part 8 — Third-party Packages and Debug Toolbar
10. Reusable Django Apps and Python Packaging
11. First Contribution to Django
12. Important Commands and Practice Project

---

1. Django Introduction

1.1 What is Django?

Django is a high-level, Python-based web framework used to build secure, scalable, and maintainable web applications.

Django can be used to build:

- Blog websites
- E-commerce websites
- REST APIs
- Learning Management Systems
- Polls and voting applications
- Inventory management systems
- Job portals

Django provides built-in authentication, ORM, URL routing, forms, an admin panel, and security features.

1.2 What is a Django Project?

A Django project is a collection of configuration files and one or more Django apps that together make up a website or web application.

```
ecommerce/
├── manage.py
├── config/
├── products/
├── orders/
├── payments/
└── users/
```

1.3 What is a Django App?

A Django app is a component responsible for a specific functionality within a project.

Examples:

- `products` — Product management
- `orders` — Order management
- `payments` — Payment processing
- `users` — User management
- `polls` — Polls and voting

1.4 Django Architecture: MVT

Django generally follows the Model–View–Template (MVT) architecture.

Component	Responsibility
Model	Defines data structures and database operations

View	Processes requests and generates responses
Template	Handles HTML presentation
URLconf	Connects URLs to views

Request flow:

```
Browser
   |
   v
URL Dispatcher
   |
   v
View
   |
   +------> Model / ORM ------> Database
   |
   +------> Template
   |
   v
HTTP Response
```

Not every request requires a database query or template rendering.

---

2. Part 1 — Creating a Django Project and App

2.1 Create a Virtual Environment

Windows:

```
python -m venv .venv
.venv\Scripts\activate
```

Linux/macOS:

```
python3 -m venv .venv
source .venv/bin/activate
```

A virtual environment keeps a project's Python dependencies isolated from other projects.

2.2 Install Django

```
python -m pip install Django
```

Check the installed version:

```
python -m django --version
```

Display package information:

```
python -m pip show Django
```

Version note: Not every Django release supports every Python version. Always check the compatibility documentation for the specific release you intend to use.

2.3 Create a Django Project

```
django-admin startproject mysite .
```

Project structure:

```
project/
├── manage.py
├── db.sqlite3
└── mysite/
    ├── __init__.py
    ├── settings.py
    ├── urls.py
    ├── asgi.py
    └── wsgi.py
```

The `db.sqlite3` file is generally created after database migrations are applied.

Important Files

File	Responsibility
`manage.py`	Runs Django management commands
`settings.py`	Contains project configuration
`urls.py`	Defines URL routing
`asgi.py`	ASGI entry point
`wsgi.py`	WSGI entry point

2.4 Run the Development Server

```
python manage.py runserver
```

Open the following URL in your browser:

```
http://127.0.0.1:8000/
```

The development server is intended for development only. Use an appropriate deployment setup in production.

2.5 Create an App

```
python manage.py startapp polls
```

App structure:

```
polls/
├── __init__.py
├── admin.py
├── apps.py
├── migrations/
│   └── __init__.py
├── models.py
├── tests.py
└── views.py
```

Create additional files and directories, such as `urls.py`, `templates/`, and `static/`, when needed.

2.6 Register the App

In `mysite/settings.py`:

```
INSTALLED_APPS = [
    "polls.apps.PollsConfig",

    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
]
```

Do not remove the existing required entries from `INSTALLED_APPS`.

In `polls/apps.py`:

```
from django.apps import AppConfig


class PollsConfig(AppConfig):

```

```
        default_auto_field = "django.db.models.BigAutoField"
        name = "polls"
```

2.7 Create the First View

In `polls/views.py`:

```
from django.http import HttpResponse


def index(request):
    return HttpResponse(
        "Hello, world. You're at the polls index."
    )
```

`HttpResponse` creates an HTTP response.

2.8 Create App URLs

In `polls/urls.py`:

```
from django.urls import path
from . import views

app_name = "polls"

urlpatterns = [
    path("", views.index, name="index"),
]
```

2.9 Connect App URLs

In `mysite/urls.py`:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("polls/", include("polls.urls")),
    path("admin/", admin.site.urls),
]
```

Open:

```
http://127.0.0.1:8000/polls/
```

Expected output:

```
Hello, world. You're at the polls index.
```

---

3. Part 2 — Database, Models, ORM and Admin

3.1 Database Configuration

The default SQLite configuration generally looks like this:

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

Django also supports database backends such as PostgreSQL, MySQL, and Oracle.

3.2 Apply Migrations

```
python manage.py migrate
```

Check the project configuration:

```
python manage.py check
```

3.3 Create Models

In `polls/models.py`:

```
import datetime

from django.db import models
from django.utils import timezone


class Question(models.Model):
    question_text = models.CharField(max_length=200)
    pub_date = models.DateTimeField("date published")

    def __str__(self):
```

```
            return self.question_text

        def was_published_recently(self):
            now = timezone.now()
            return now - datetime.timedelta(days=1) <= self.pub_date <= now


    class Choice(models.Model):
        question = models.ForeignKey(
            Question,
            on_delete=models.CASCADE,
        )
        choice_text = models.CharField(max_length=200)
        votes = models.IntegerField(default=0)

        def __str__(self):
            return self.choice_text
```

Model Fields

Field	Responsibility
`CharField`	Stores text
`DateTimeField`	Stores date and time
`IntegerField`	Stores integers
`ForeignKey`	Defines a many-to-one relationship

`default=0`	Sets the default vote count to zero
`CASCADE`	Deletes related `Choice` objects when the parent `Question` is deleted

The `was_published_recently()` method returns `True` for questions published within the previous 24 hours, provided their publication time is not in the future.

3.4 Create and Apply Migrations

```
python manage.py makemigrations polls
python manage.py migrate
```

Differences:

- `makemigrations` — Creates migration files.
- `migrate` — Applies migrations to the database.
- `showmigrations` — Displays migration status.
- `sqlmigrate` — Displays the SQL associated with a migration.

3.5 Django ORM

ORM stands for Object-Relational Mapping. It allows you to interact with a database using Python code.

Raw SQL:

```
SELECT * FROM polls_question;
```

Django ORM:

```
Question.objects.all()
```

3.6 Django Shell

Open the Django shell:

```
python manage.py shell
```

Create a question:

```
from django.utils import timezone
from polls.models import Question, Choice

q = Question(
    question_text="What's new?",
    pub_date=timezone.now(),
)

q.save()
```

Access the object's ID:

```
q.id
q.pk
```

Update an object:

```
q.question_text = "What's up?"
q.save()
```

Retrieve objects:

```
Question.objects.all()
Question.objects.filter(id=1)
Question.objects.get(pk=1)
Question.objects.order_by("-pub_date")
Question.objects.count()
```

The `get()` method expects exactly one matching object. If no object exists, Django raises `DoesNotExist`. If multiple objects match, it raises `MultipleObjectsReturned`.

3.7 Important QuerySet Methods

Method	Responsibility
`all()`	Returns all matching objects
`filter()`	Returns objects matching specified conditions
`exclude()`	Excludes objects matching specified conditions
`get()`	Retrieves a single object
`order_by()`	Sorts results
`count()`	Returns the number of matching objects
`exists()`	Checks whether matching

	objects exist
`create()`	Creates and saves an object
`update()`	Updates matching records
`delete()`	Deletes matching records
`first()`	Returns the first object or `None`
`last()`	Returns the last object or `None`

Example:

```
Question.objects.filter(
    question_text__startswith="What"
).count()
```

QuerySets are generally lazy, meaning database queries may not execute until the results are needed.

3.8 Create Choices

```
q.choice_set.create(
    choice_text="Not much",
```

```
        votes=0,
    )

    q.choice_set.create(
        choice_text="The sky",
        votes=0,
    )

    q.choice_set.create(
        choice_text="Just hacking again",
        votes=0,
    )
```

Retrieve related choices:

```
q.choice_set.all()
q.choice_set.count()
```

3.9 Create a Superuser

```
python manage.py createsuperuser
```

Admin URL:

```
http://127.0.0.1:8000/admin/
```

3.10 Register Models

In `polls/admin.py`:

```
from django.contrib import admin
from .models import Choice, Question

admin.site.register(Question)
admin.site.register(Choice)
```

---

4. Part 3 — Views and Templates

4.1 URL Parameters

In `polls/views.py`:

```
from django.http import HttpResponse
```

```
def detail(request, question_id):
    return HttpResponse(
        f"You're looking at question {question_id}."
    )


def results(request, question_id):
    return HttpResponse(
        f"You're looking at results of question {question_id}."
    )
```

In `polls/urls.py`:

```
from django.urls import path
from . import views

app_name = "polls"

urlpatterns = [
    path("", views.index, name="index"),
    path("<int:question_id>/", views.detail, name="detail"),
    path(
        "<int:question_id>/results/",
        views.results,
        name="results",
    ),
]
```

The `<int:question_id>` converter captures an integer from the URL and passes it to the view.

4.2 Template Directory

```
polls/
└── templates/
    └── polls/
        ├── index.html
        ├── detail.html
        └── results.html
```

4.3 Index Template

In `polls/templates/polls/index.html`:

```
{% if latest_question_list %}
    <ul>
        {% for question in latest_question_list %}
            <li>
                <a href="{% url 'polls:detail' question.id %}">
                    {{ question.question_text }}
                </a>
            </li>
        {% endfor %}
    </ul>
{% else %}
    <p>No polls are available.</p>
{% endif %}
```

Template Syntax

Syntax	Responsibility
`{{ variable }}`	Outputs a variable
`{% if %}`	Defines a condition
`{% for %}`	Iterates over a collection
`{% url %}`	Generates a named URL
`{% endif`	Ends an `if` block

`%}`	
`{% endfor %}`	Ends a `for` loop

Django templates generally escape HTML output automatically. Be careful when using the `safe` filter or disabling automatic escaping with `autoescape off`.

4.4 Render a Template

In `polls/views.py`:

```
from django.shortcuts import render
from .models import Question


def index(request):
    latest_question_list = Question.objects.order_by(
        "-pub_date"
    )[:5]

    context = {
        "latest_question_list": latest_question_list,
    }

    return render(
        request,
        "polls/index.html",
        context,
    )
```

4.5 Detail View with 404

```
from django.shortcuts import get_object_or_404, render
from .models import Question


def detail(request, question_id):
    question = get_object_or_404(
        Question,
```

```
            pk=question_id,
        )

        return render(
            request,
            "polls/detail.html",
            {"question": question},
        )
```

`get_object_or_404()` returns the requested object or raises an HTTP 404 exception if the object does not exist.

4.6 Detail Template

In `polls/templates/polls/detail.html`:

```
<h1>{{ question.question_text }}</h1>

<ul>
    {% for choice in question.choice_set.all %}
        <li>{{ choice.choice_text }}</li>
    {% empty %}
        <li>No choices are available.</li>
    {% endfor %}
</ul>
```

4.7 URL Namespaces

In `polls/urls.py`:

```
app_name = "polls"
```

In a template:

```
<a href="{% url 'polls:detail' question.id %}">
    {{ question.question_text }}
</a>
```

In Python:

```
from django.urls import reverse

url = reverse("polls:detail", args=[question.id])
```

Named URLs reduce hardcoded URLs and make URL configuration easier to maintain.

---

5. Part 4 — Forms, Voting and Generic Views

5.1 Voting Form

In `polls/templates/polls/detail.html`:

```
<form action="{% url 'polls:vote' question.id %}" method="post">
    {% csrf_token %}

    <fieldset>
        <legend>
            <h1>{{ question.question_text }}</h1>
        </legend>

        {% if error_message %}
            <p><strong>{{ error_message }}</strong></p>
        {% endif %}

        {% for choice in question.choice_set.all %}
            <input
                type="radio"
                name="choice"
                id="choice{{ forloop.counter }}"
                value="{{ choice.id }}"
            >

            <label for="choice{{ forloop.counter }}">
                {{ choice.choice_text }}
            </label>
            <br>
        {% empty %}
            <p>No choices are available.</p>
        {% endfor %}
    </fieldset>

    {% if question.choice_set.exists %}
        <input type="submit" value="Vote">
    {% endif %}
</form>
```

The `{% csrf_token %}` tag inserts the token needed for CSRF protection.

5.2 Vote View

In `polls/views.py`:

```
from django.db.models import F
from django.http import HttpResponseRedirect
from django.shortcuts import get_object_or_404, render
from django.urls import reverse

from .models import Choice, Question


def vote(request, question_id):
    question = get_object_or_404(
        Question,
        pk=question_id,
    )

    if request.method != "POST":
        return HttpResponseRedirect(
            reverse("polls:detail", args=[question.id])
        )

    try:
        selected_choice = question.choice_set.get(
            pk=request.POST["choice"]
        )
    except (KeyError, Choice.DoesNotExist):
        return render(
            request,
            "polls/detail.html",
            {
                "question": question,
                "error_message": "You didn't select a valid choice.",
            },
            status=400,
        )

    selected_choice.votes = F("votes") + 1
    selected_choice.save(update_fields=["votes"])
```

```
        return HttpResponseRedirect(
            reverse(
                "polls:results",
                args=[question.id],
            )
        )
```

Important Concepts

- Use POST requests for state-changing operations.
- CSRF tokens help protect form submissions against cross-site request forgery.
- `question.choice_set.get()` ensures that the selected choice belongs to the specified question.
- `F("votes") + 1` increments the current database value by one.
- Additional business logic is required to prevent the same user from voting repeatedly.

5.3 Results View

```
def results(request, question_id):
    question = get_object_or_404(
        Question,
        pk=question_id,
    )

    return render(
        request,
        "polls/results.html",
        {"question": question},
    )
```

In `polls/templates/polls/results.html`:

```
<h1>{{ question.question_text }}</h1>

<ul>
    {% for choice in question.choice_set.all %}
        <li>
            {{ choice.choice_text }} —
            {{ choice.votes }}
            vote{{ choice.votes|pluralize }}
```

```
            </li>
        {% empty %}
            <li>No choices are available.</li>
        {% endfor %}
    </ul>

    <a href="{% url 'polls:detail' question.id %}">
        Vote again?
    </a>
```

5.4 Configure URLs

In `polls/urls.py`:

```
    from django.urls import path
    from . import views

    app_name = "polls"

    urlpatterns = [
        path("", views.index, name="index"),
        path("<int:question_id>/", views.detail, name="detail"),
        path(
            "<int:question_id>/results/",
            views.results,
            name="results",
        ),
        path(
            "<int:question_id>/vote/",
            views.vote,
            name="vote",
        ),
    ]
```

5.5 Generic Views

Generic views are reusable class-based views designed to handle common web development tasks.

Generic View	Responsibility

`ListView`	Displays a list of objects
`DetailView`	Displays a single object
`CreateView`	Creates an object
`UpdateView`	Updates an object
`DeleteView`	Deletes an object

Replace the index, detail, and results views in `polls/views.py` with:

```
from django.views import generic
from .models import Question


class IndexView(generic.ListView):
    template_name = "polls/index.html"
    context_object_name = "latest_question_list"

    def get_queryset(self):
        return Question.objects.order_by("-pub_date")[:5]


class DetailView(generic.DetailView):
    model = Question
    template_name = "polls/detail.html"


class ResultsView(generic.DetailView):
```

```
        model = Question
        template_name = "polls/results.html"
```

URLs:

```
from django.urls import path
from . import views

app_name = "polls"

urlpatterns = [
    path("", views.IndexView.as_view(), name="index"),
    path("<int:pk>/", views.DetailView.as_view(), name="detail"),
    path(
        "<int:pk>/results/",
        views.ResultsView.as_view(),
        name="results",
    ),
    path(
        "<int:question_id>/vote/",
        views.vote,
        name="vote",
    ),
]
```

Here, `DetailView` uses `pk`, while the `vote` function uses `question_id`. URL parameter names must match the parameters expected by the corresponding view.

---

6. Part 5 — Automated Testing

6.1 What Is Automated Testing?

Automated testing is the process of checking code automatically to verify that it behaves as expected.

Benefits include:

- Identifying bugs
- Preventing regressions
- Making refactoring safer
- Documenting expected behavior

6.2 Model Tests

In `polls/tests.py`:

```
import datetime

from django.test import TestCase
from django.utils import timezone

from .models import Question


class QuestionModelTests(TestCase):

    def test_future_question_is_not_recent(self):
        future_time = timezone.now() + datetime.timedelta(days=30)

        question = Question(
            question_text="Future question",
            pub_date=future_time,
        )

        self.assertIs(
            question.was_published_recently(),
            False,
        )

    def test_old_question_is_not_recent(self):
        old_time = timezone.now() - datetime.timedelta(
            days=1,
            seconds=1,
        )

        question = Question(
            question_text="Old question",
            pub_date=old_time,
        )

        self.assertIs(
            question.was_published_recently(),
            False,
        )
```

```
        def test_recent_question_is_recent(self):
            recent_time = timezone.now() - datetime.timedelta(hours=1)

            question = Question(
                question_text="Recent question",
                pub_date=recent_time,
            )

            self.assertIs(
                question.was_published_recently(),
                True,
            )
```

6.3 Run Tests

Run all tests:

```
python manage.py test
```

Run tests for the polls app:

```
python manage.py test polls
```

Run a specific test class:

```
python manage.py test polls.tests.QuestionModelTests
```

6.4 Important Assertions

Assertion	Responsibility
`assertEqual(a, b)`	Checks whether values are equal
`assertNotEqual(a, b)`	Checks whether

	values are different
`assertTrue(x)`	Checks whether a value is truthy
`assertFalse(x)`	Checks whether a value is falsy
`assertIsNone(x)`	Checks whether a value is `None`
`assertContains(response, text)`	Checks whether the response contains text
`assertRaises(Exception)`	Checks whether an exception is raised

6.5 Test Client

```
from django.test import TestCase
from django.urls import reverse
```

```
class QuestionIndexViewTests(TestCase):

    def test_no_questions(self):
        response = self.client.get(
            reverse("polls:index")
        )

        self.assertEqual(response.status_code, 200)
        self.assertContains(
            response,
            "No polls are available.",
        )
```

Test that an invalid question returns a 404:

```
def test_invalid_question_returns_404(self):
    response = self.client.get(
        reverse("polls:detail", args=[99999])
    )

    self.assertEqual(response.status_code, 404)
```

Test an invalid vote:

```
def test_vote_requires_valid_choice(self):
    from django.utils import timezone
    from polls.models import Question

    question = Question.objects.create(
        question_text="Favourite language?",
        pub_date=timezone.now(),
    )

    response = self.client.post(
        reverse("polls:vote", args=[question.pk]),
        {},
    )

    self.assertEqual(response.status_code, 400)
```

This final test is consistent with the implementation of the earlier vote view.

---

7. Part 6 — Static Files

7.1 What Are Static Files?

Static files are assets such as CSS, JavaScript, images, fonts, and icons.

Directory structure:

```
polls/
├── static/
│   └── polls/
│       ├── style.css
│       └── images/
│           └── background.png
└── templates/
    └── polls/
        └── index.html
```

7.2 CSS File

In `polls/static/polls/style.css`:

```
body {
    background-color: white;
    font-family: Arial, sans-serif;
}

li a {
    color: green;
}
```

7.3 Load CSS in a Template

In `polls/templates/polls/index.html`:

```
{% load static %}

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Polls</title>

    <link
```

```
            rel="stylesheet"
            href="{% static 'polls/style.css' %}"
        >
    </head>
    <body>
        <h1>Polls</h1>
    </body>
    </html>
```

7.4 Static Settings

For development:

```
STATIC_URL = "static/"
```

For production, you will generally configure:

```
STATIC_ROOT = BASE_DIR / "staticfiles"
```

If you have additional static source directories:

```
STATICFILES_DIRS = [
    BASE_DIR / "static",
]
```

Only add `STATICFILES_DIRS` when you have additional static source directories, such as a project-level `static/` directory.

Collect static files:

```
python manage.py collectstatic
```

Differences:

- `STATIC_URL` — The URL prefix for static files.
- `STATIC_ROOT` — The output directory used by `collectstatic`.
- `STATICFILES_DIRS` — Additional directories containing static source files.

Production environments require appropriate static file hosting or web server configuration.

---

8. Part 7 — Django Admin Customization

8.1 Customize Fields

In `polls/admin.py`:

```
from django.contrib import admin
from .models import Question


@admin.register(Question)
class QuestionAdmin(admin.ModelAdmin):
    fields = ["pub_date", "question_text"]
```

8.2 Fieldsets

```
@admin.register(Question)
class QuestionAdmin(admin.ModelAdmin):
    fieldsets = [
        (
            None,
            {
                "fields": ["question_text"],
            },
        ),
        (
            "Date information",
            {
                "fields": ["pub_date"],
                "classes": ["collapse"],
            },
        ),
    ]
```

Avoid configuring both `fields` and `fieldsets` together in the same `ModelAdmin` when defining the field layout.

8.3 Inline Choices

```
from django.contrib import admin
from .models import Choice, Question


class ChoiceInline(admin.TabularInline):
    model = Choice
    extra = 3
```

```
@admin.register(Question)
class QuestionAdmin(admin.ModelAdmin):
    fieldsets = [
        (
            None,
            {
                "fields": ["question_text"],
            },
        ),
        (
            "Date information",
            {
                "fields": ["pub_date"],
                "classes": ["collapse"],
            },
        ),
    ]

    inlines = [ChoiceInline]
```

`TabularInline` displays related objects in a table-style form, while `StackedInline` uses a stacked form layout.

8.4 List Display, Search, and Filters

```
@admin.register(Question)
class QuestionAdmin(admin.ModelAdmin):
    list_display = [
        "question_text",
        "pub_date",
        "was_published_recently",
    ]
    list_filter = ["pub_date"]
    search_fields = ["question_text"]
    date_hierarchy = "pub_date"
    ordering = ["-pub_date"]
```

Option	Responsibility
`list_display`	Defines columns
`list_filter`	Adds sidebar filters
`search_fields`	Configures the search box
`date_hierarchy`	Adds date-based navigation
`ordering`	Defines default sorting

8.5 Admin Site Titles

In `mysite/urls.py`:

```
from django.contrib import admin
from django.urls import include, path

admin.site.site_header = "Polls Administration"
admin.site.site_title = "Polls Admin"
admin.site.index_title = "Welcome to Polls Admin"

urlpatterns = [
    path("polls/", include("polls.urls")),
```

```
    path("admin/", admin.site.urls),
]
```

---

9. Part 8 — Third-party Packages and Debug Toolbar

9.1 Third-party Packages

Third-party packages are reusable libraries developed outside Django's core codebase.

Examples:

- Django REST Framework — REST APIs
- Django Debug Toolbar — Development debugging
- `django-filter` — Filtering
- Pillow — Image processing
- Celery — Background tasks

Before installing a package, check its compatibility, maintenance status, and security.

9.2 Install Django Debug Toolbar

```
python -m pip install django-debug-toolbar
```

Verify the installation:

```
python -m pip show django-debug-toolbar
```

9.3 Add to INSTALLED_APPS

In `mysite/settings.py`:

```
INSTALLED_APPS = [
    "debug_toolbar",
    "polls.apps.PollsConfig",

    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
]
```

9.4 Add Middleware

In `mysite/settings.py`:

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",

    "debug_toolbar.middleware.DebugToolbarMiddleware",
]
```

Do not remove existing required middleware. Check the package installation instructions for the recommended middleware placement.

9.5 Configure Internal IPs

```
INTERNAL_IPS = [
    "127.0.0.1",
]
```

9.6 Add Debug Toolbar URLs

In `mysite/urls.py`:

```
from django.conf import settings
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("polls/", include("polls.urls")),
    path("admin/", admin.site.urls),
]

if settings.DEBUG:
    import debug_toolbar

    urlpatterns = [
        path("__debug__/", include(debug_toolbar.urls)),
    ] + urlpatterns
```

9.7 Run and Inspect

```
python manage.py check
python manage.py runserver
```

Open:

```
http://127.0.0.1:8000/polls/
```

The SQL panel in Django Debug Toolbar can be used to inspect database queries.

Security: Never expose Debug Toolbar in production. Set `DEBUG=False` and use appropriate deployment settings in production environments.

---

10. Reusable Django Apps and Python Packaging

10.1 What Is a Reusable App?

A reusable Django app is an app that can be installed and used in multiple Django projects.

```
Project A
└── polls

Project B
└── polls
```

10.2 Python Package vs. Django App

Python Package	Django App
Organizes and reuses Python code	Implements specific functionality within a Django project

Example: `requests`	Example: `polls`
Contains Python modules	May contain models, views, URLs, templates, and other Django components

A Django app can also be a Python package.

10.3 Package Structure

```
django-polls/
├── django_polls/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── migrations/
│   ├── models.py
│   ├── tests.py
│   ├── urls.py
│   ├── views.py
│   ├── static/
│   └── templates/
├── README.rst
├── LICENSE
├── MANIFEST.in
└── pyproject.toml
```

Here:

- `django-polls` — Distribution package name.
- `django_polls` — Python import package name.

10.4 AppConfig

In `django_polls/apps.py`:

```
from django.apps import AppConfig


class PollsConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "django_polls"
    label = "polls"
```

`name` specifies the Python package path, while `label` specifies the Django app label.

Keeping an existing app label may help preserve migration and database compatibility. However, migration history and the existing database must be checked separately.

10.5 README.rst

```
============
django-polls
============

django-polls is a Django app to conduct web-based polls.

Quick start
-----------

1. Add "django_polls" to INSTALLED_APPS.

2. Include the app URLconf:

   path("polls/", include("django_polls.urls"))

3. Run migrations:

   python manage.py migrate

4. Create a superuser:
```

```
   python manage.py createsuperuser

5. Start the development server and visit /polls/.
```

10.6 License

Include the complete text of your chosen open-source license in the `LICENSE` file. Creating a file with that name alone does not establish a license.

10.7 pyproject.toml

The following is an example of a setuptools-based packaging configuration:

```
[build-system]
requires = ["setuptools"]
build-backend = "setuptools.build_meta"

[project]
name = "django-polls"
version = "0.1.0"
description = "A Django app to conduct web-based polls."
readme = "README.rst"
requires-python = ">=3.12"
license = "BSD-3-Clause"
dependencies = [
    "Django>=5.2,<6.0",
]

[tool.setuptools.packages.find]
include = ["django_polls*"]

[tool.setuptools.package-data]
django_polls = [
    "templates/**/*.html",
    "static/**/*",
]
```

Version note: Python `>=3.12` and Django `>=5.2,<6.0` are example compatibility targets. If you intend to support other Django or Python versions, adjust the dependency constraints and test matrix accordingly.

10.8 MANIFEST.in

```
recursive-include django_polls/templates *
recursive-include django_polls/static *
```

`MANIFEST.in` primarily helps include files in the source distribution. You should also verify that templates and static assets are included in the built wheel.

10.9 Build the Package

Install the build tool:

```
python -m pip install --upgrade build
```

From the package directory, run:

```
python -m build
```

The following files are typically generated:

```
dist/
├── django_polls-0.1.0.tar.gz
└── django_polls-0.1.0-py3-none-any.whl
```

- `.tar.gz` — Source Distribution
- `.whl` — Wheel Distribution

10.10 Install and Test

Install the wheel:

```
python -m pip install dist/django_polls-0.1.0-py3-none-any.whl
```

In the consumer project's `settings.py`:

```
INSTALLED_APPS = [
    "django_polls.apps.PollsConfig",
    # Existing Django apps...
]
```

In `mysite/urls.py`:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("polls/", include("django_polls.urls")),
```

```
        path("admin/", admin.site.urls),
    ]
```

Then run:

```
python manage.py check
python manage.py migrate
python manage.py test django_polls
```

For local development, use:

```
python -m pip install -e .
```

Editable installation makes it easier to modify and test the source code without repeatedly reinstalling the package.

---

11. First Contribution to Django

11.1 What Is Open-source Contribution?

Open-source contribution means improving an open-source project through code, tests, documentation, bug fixes, or other useful changes.

Types of Django contributions include:

- Bug fixes
- New features
- Automated tests
- Documentation improvements
- Refactoring
- Performance improvements

11.2 Check Git Version

```
git --version
```

11.3 Fork and Clone Django

Django repository:

https://github.com/django/django

After forking the repository on GitHub:

```
git clone https://github.com/YOUR_USERNAME/django.git
cd django
```

Replace `YOUR_USERNAME` with your GitHub username.

11.4 Create a Virtual Environment

Linux/macOS:

```
python3 -m venv ~/.virtualenvs/djangodev
source ~/.virtualenvs/djangodev/bin/activate
```

Windows PowerShell:

```
py -m venv "$HOME\.virtualenvs\djangodev"
& "$HOME\.virtualenvs\djangodev\Scripts\Activate.ps1"
```

From the Django repository root:

```
python -m pip install -e .
```

11.5 Install Test Dependencies

If your checkout contains `tests/requirements/py3.txt`, run:

```
python -m pip install -r tests/requirements/py3.txt
```

Version note: Testing setup and dependency file locations may vary between Django branches and releases. Follow the contribution documentation for your specific checkout.

11.6 Run Tests

From the Django repository root:

```
python tests/runtests.py
```

Run the shortcuts-related tests:

```
python tests/runtests.py shortcuts
```

Common test output symbols:

- `.` — Passed
- `F` — Failure
- `E` — Error
- `s` — Skipped

- `x` — Expected failure

11.7 Create a Feature Branch

```
git checkout -b ticket_99999
```

`ticket_99999` is an example branch name. Use an appropriate name based on the actual ticket or task.

11.8 Test-Driven Development

The typical TDD workflow is:

1. Write a test.
2. Run the test and observe the failure.
3. Implement the required code.
4. Run the test again.
5. Refactor the code.
6. Run the tests again.

11.9 Hypothetical make_toast() Example

The following example is fictional and intended for learning purposes. `make_toast()` is not a built-in Django function.

Test:

```
from django.shortcuts import make_toast
from django.test import SimpleTestCase


class MakeToastTests(SimpleTestCase):

    def test_make_toast(self):
        self.assertEqual(make_toast(), "toast")
```

Example implementation:

```
def make_toast():
    return "toast"
```

Run the test:

```
python tests/runtests.py shortcuts
```

For an actual Django contribution, follow an accepted issue, the project's design decisions, and the maintainers' review process.

11.10 Documentation

Django documentation generally uses reStructuredText (`.rst`).

Fictional documentation example:

```
``make_toast()``
================

.. function:: make_toast()

   Returns ``'toast'``.
```

When changing a real public API, update the appropriate documentation and release notes when required.

Important: Do not treat the tutorial's `make_toast()` example as an actual Django 6.2 feature.

11.11 Review and Commit

Stage changes:

```
git add --all
git diff --cached
```

Commit changes:

```
git commit -m "Add the new shortcut function"
```

For a real contribution, write a commit message that follows the project's guidelines.

11.12 Push and Create a Pull Request

```
git push origin ticket_99999
```

Then create a pull request on GitHub from your branch.

Include the following information in the pull request:

- What problem has been solved?
- What changes have been made?
- Which tests have been run?

- Has the documentation been updated?

Maintainers may review the code and request additional changes.

11.13 Official Resources

- Django Ticket Tracker: https://code.djangoproject.com/
- Django GitHub: https://github.com/django/django
- Contribution Documentation: https://docs.djangoproject.com/en/stable/internals/contributing/
- Testing Documentation: https://docs.djangoproject.com/en/stable/topics/testing/

---

12. Important Commands and Practice Project

12.1 Django Commands

```
# Create a virtual environment
python -m venv .venv

# Install Django
python -m pip install Django

# Check Django version
python -m django --version

# Create a project
django-admin startproject mysite .

# Create an app
python manage.py startapp polls

# Run the development server
python manage.py runserver

# Check project configuration
python manage.py check

# Create migrations
python manage.py makemigrations

# Apply migrations
python manage.py migrate
```

```
# Show migrations
python manage.py showmigrations

# Show SQL for a migration
python manage.py sqlmigrate polls 0001

# Open the Django shell
python manage.py shell

# Create a superuser
python manage.py createsuperuser

# Run tests
python manage.py test

# Collect static files
python manage.py collectstatic
```

12.2 CRUD Using Django ORM

Create

```
from django.utils import timezone
from polls.models import Question

question = Question.objects.create(
    question_text="Which language do you prefer?",
    pub_date=timezone.now(),
)
```

Read

```
Question.objects.all()
Question.objects.get(pk=1)
```

Update

```
question = Question.objects.get(pk=1)

question.question_text = "Which programming language do you prefer?"
question.save()
```

Delete

```
question = Question.objects.get(pk=1)
question.delete()
```

Warning: `delete()` permanently removes data from the database. Before working with important data, verify your backups and deletion rules.

12.3 Common Errors

Error	Possible Cause
`ModuleNotFoundError`	Incorrect environment or missing package
`No module named polls`	Incorrect app path or `INSTALLED_APPS` configuration
`TemplateDoesNotExist`	Incorrect template path
`NoReverseMatch`	Incorrect URL name, namespace, or

	argument s
`no such table`	Migrations have not been applied
`DoesNot Exist`	No matching object was found
`CSRF verification failed`	CSRF token or configurati on problem
CSS not loading	Incorrect static path or configurati on
Debug Toolbar missing	Middlewar e, IP, or `DEBUG` configurati on issue
Package import error	Packaging or installatio n issue

12.4 Practice Project Checklist

- Create a virtual environment and Django project.
- Create and register the polls app.
- Create the Question and Choice models.
- Apply migrations.
- Configure Django Admin.
- Create the question index page.
- Create the question detail page.
- Implement the voting form.
- Create the results page.
- Write automated tests.
- Add CSS and static files.
- Configure admin search and filters.
- Inspect database queries using Debug Toolbar.
- Build a reusable Django package.
- Install and test the package in another project.
- Practice the open-source contribution workflow.

12.5 Recommended Learning Order

1. Django ORM and advanced queries
2. Forms and validation
3. Authentication and permissions
4. Class-based views
5. Django REST Framework
6. PostgreSQL and database design
7. Automated testing
8. Git and GitHub
9. Docker and deployment
10. Security, performance, and caching
11. Open-source contribution

Official Resources

- Django Tutorial: https://docs.djangoproject.com/en/stable/intro/tutorial01/
- Models and Database: https://docs.djangoproject.com/en/stable/topics/db/
- Testing: https://docs.djangoproject.com/en/stable/topics/testing/

- Static Files: https://docs.djangoproject.com/en/stable/howto/static-files/
- Python Packaging: https://packaging.python.org/en/latest/tutorials/packaging-projects/

Final Note: Django versions, Python compatibility, package dependencies, and contribution workflows may change over time. Before starting a project, consult the official documentation compatible with your installed Django version.